

Notions avancées en POO

1. L'encapsulation

L'encapsulation est un mécanisme de restriction de l'accès direct aux composants d'un objet, ce qui protège l'intégrité des données de l'objet en empêchant des interférences extérieures ou une utilisation incorrecte.

- **Protection des données :**

Les attributs d'un objet sont cachés et seulement accessibles via des méthodes définies dans la classe (souvent appelées getters et setters). Cela assure que les données internes ne peuvent pas être changées arbitrairement, imposant un contrôle strict sur la manière dont les données essentielles sont modifiées et consultées.

- **Contrôle de l'accès :**

L'encapsulation utilise des modificateurs d'accès tels que public, private, et protected pour contrôler comment les membres de données sont visibles à l'extérieur de la classe. Les méthodes publiques fournissent une interface avec l'extérieur, tandis que les détails internes restent privés, permettant à la classe de garantir qu'aucun comportement erroné ne soit induit par des changements inattendus dans son état interne.

- **Cohérence et validation :**

En encapsulant les données, les classes peuvent forcer des règles de validation sur les données entrantes, garantissant que l'objet reste dans un état valide. Par exemple, si une classe `CompteBancaire` a un attribut `solde`, le setter pour ce solde peut vérifier que les fonds ne sont pas définis à une valeur négative.

L'encapsulation et le polymorphisme sont des piliers de la POO qui permettent de construire des systèmes plus fiables, modulaires et évolutifs. Ces mécanismes aident à gérer la complexité et à augmenter la réutilisabilité du code, tout en améliorant la maintenance et la flexibilité des logiciels.

Exemple sans encapsulation en PHP

```
<?php
```

```
class Dog {
```

```

public $name;
public $breed;

public function __construct($name, $breed) {
    $this->name = $name;
    $this->breed = $breed;
}

public function speak() {
    return "Woof!";
}
}

$dog = new Dog("Buddy", "Golden Retriever");
echo $dog->name . " says " . $dog->speak() . "\n"; // Buddy says Woof!
echo "Breed: " . $dog->breed . "\n"; // Breed: Golden Retriever

// Direct access to properties
$dog->name = "Max";
echo $dog->name . " says " . $dog->speak() . "\n"; // Max says Woof!

```

Exemple avec encapsulation en PHP

```

<?php

class Dog {
    private $name;
    private $breed;

    public function __construct($name, $breed) {
        $this->setName($name);
        $this->setBreed($breed);
    }

    public function setName($name) {
        if (!empty($name)) {
            $this->name = $name;
        } else {
            throw new Exception("Name cannot be empty");
        }
    }

    public function getName() {
        return $this->name;
    }
}

```

```

    public function setBreed($breed) {
        if (!empty($breed)) {
            $this->breed = $breed;
        } else {
            throw new Exception("Breed cannot be empty");
        }
    }

    public function getBreed() {
        return $this->breed;
    }

    public function speak() {
        return "Woof!";
    }
}

$dog = new Dog("Buddy", "Golden Retriever");
echo $dog->getName() . " says " . $dog->speak() . "\n"; // Buddy says Woof!
echo "Breed: " . $dog->getBreed() . "\n"; // Breed: Golden Retriever

// Trying to change the name directly will not work
// $dog->name = "Max"; // Error
$dog->setName("Max");
echo $dog->getName() . " says " . $dog->speak() . "\n"; // Max says Woof!

```

Comparaison avec et sans encapsulation

1. Sans encapsulation :

- Les propriétés de la classe sont accessibles directement, ce qui peut mener à des modifications involontaires ou incorrectes des données internes.
- Le code est moins sécurisé et moins maintenable.

2. Avec encapsulation :

- Les propriétés de la classe sont privées et accessibles uniquement via des méthodes publiques (getter et setter).
- Cela permet de contrôler et de valider les modifications des données internes.
- Le code est plus sécurisé, maintenable et respecte le principe d'encapsulation de la POO.

2. Le polymorphisme

Le polymorphisme est un principe clé de la programmation orientée objet (POO) qui permet aux objets de différentes classes de répondre à la même action, chaque objet traitant l'action d'une manière appropriée à son type. Ce concept repose sur trois aspects fondamentaux : les interfaces, l'abstraction, et la concrétisation.

- **Interfaces** : En POO, une interface est une déclaration de méthodes sans implémentations, agissant comme un contrat que les classes implémentantes doivent respecter. Les interfaces permettent à des classes de types très différents de s'interfacer de manière uniforme avec le reste du système.
- **Abstraction** :
Le polymorphisme favorise l'abstraction en permettant l'utilisation de types plus généraux (comme les types de base dans une hiérarchie de classes ou les interfaces) pour manipuler des objets. Cela permet aux développeurs de programmer en termes d'interfaces ou de classes abstraites tout en laissant le système décider de l'objet spécifique à créer et à utiliser en fonction du contexte.
- **Concrétisation** :
Cela se produit lorsque des classes concrètes implémentent les méthodes définies par leurs interfaces ou classes abstraites. La concrétisation est l'acte de définir le comportement spécifique des méthodes abstraites ou des interfaces, fournissant ainsi une implémentation réelle aux opérations abstraites.

Exemple sans polymorphisme

Imaginons que nous ayons deux types de formes, un rectangle et un cercle, et que nous voulions calculer et afficher leurs aires. Sans utiliser le polymorphisme, nous devons traiter chaque forme séparément avec des fonctions spécifiques.

```
<?php

// Classe Dog
class Dog {
    public function speak() {
        return "Woof!";
    }
}

// Classe Cat
class Cat {
    public function speak() {
        return "Meow!";
    }
}
```

```
// Fonctions pour faire parler les animaux
function makeDogSpeak(Dog $dog) {
    echo "Dog says: " . $dog->speak() . "\n";
}

function makeCatSpeak(Cat $cat) {
    echo "Cat says: " . $cat->speak() . "\n";
}

// Utilisation des classes et fonctions
$dog = new Dog();
$cat = new Cat();

makeDogSpeak($dog); // Dog says: Woof!
makeCatSpeak($cat); // Cat says: Meow!
```

Exemple avec polymorphisme

Utilisons maintenant le polymorphisme pour simplifier le code. Nous allons créer une classe de base Shape avec une méthode abstraite area() que chaque sous-classe devra implémenter.

```
<?php

// Classe de base
abstract class Animal {
    // Méthode abstraite
    abstract public function speak();
}

// Classe dérivée : Dog
class Dog extends Animal {
    public function speak() {
        return "Woof!";
    }
}

// Classe dérivée : Cat
class Cat extends Animal {
    public function speak() {
        return "Meow!";
    }
}

// Utilisation du polymorphisme
function makeAnimalSpeak(Animal $animal) {
```

```

        echo get_class($animal) . " says: " . $animal-> speak() . "\n";
    }

// Utilisation des classes avec polymorphisme
$dog = new Dog();
$cat = new Cat();

makeAnimalSpeak($dog); // Dog says: Woof!
makeAnimalSpeak($cat); // Cat says: Meow!

```

Comparaison des deux approches

1. Sans polymorphisme :

- Nous devons écrire des fonctions spécifiques pour chaque type d'animal.
- Le code n'est pas facilement extensible si nous voulons ajouter de nouveaux animaux.
- La gestion de chaque animal est dispersée, rendant le code plus difficile à maintenir.

2. Avec polymorphisme :

- Nous avons une interface commune (Animal) pour tous les animaux.
- La méthode `speak()` est implémentée spécifiquement dans chaque sous-classe.
- Le code est plus propre, plus extensible et plus facile à maintenir.
- Une seule fonction `makeAnimalSpeak()` peut traiter tous les animaux.

Exemple amélioré avec une interface

Créons une interface `Speakable` qui déclare la méthode `speak()` . Ensuite, nous implémenterons cette interface dans nos classes `Dog` et `Cat` .

```

<?php

// Interface Speakable
interface Speakable {
    public function speak();
}

// Classe Dog implémentant l'interface Speakable
class Dog implements Speakable {
    public function speak() {
        return "Woof!";
    }
}

// Classe Cat implémentant l'interface Speakable
class Cat implements Speakable {

```

```

    public function speak() {
        return "Meow!";
    }
}

// Utilisation du polymorphisme avec l'interface
function makeAnimalSpeak(Speakable $animal) {
    echo get_class($animal) . " says: " . $animal->speak() . "\n";
}

// Utilisation des classes avec polymorphisme et interface
$dog = new Dog();
$cat = new Cat();

makeAnimalSpeak($dog); // Dog says: Woof!
makeAnimalSpeak($cat); // Cat says: Meow!

```

Comparaison avec et sans interface

1. Sans interface :

- Le polymorphisme est basé sur une classe abstraite.
- Les classes dérivées doivent hériter de cette classe abstraite.

2. Avec interface :

- Le polymorphisme est basé sur une interface.
- Les classes peuvent implémenter plusieurs interfaces, ce qui n'est pas possible avec les classes abstraites.
- L'interface définit un contrat clair que toutes les classes doivent suivre.

Avantages d'utiliser une interface

- **Flexibilité** : Une classe peut implémenter plusieurs interfaces, permettant ainsi un polymorphisme plus flexible.
- **Clarté** : L'interface définit un contrat clair que toutes les classes doivent suivre, ce qui rend le code plus lisible et compréhensible.
- **Modularité** : Facilite l'ajout de nouvelles classes qui respectent le même contrat sans modifier les classes existantes.

Exemple combinant interface et abstraction

```
<?php
```

```
// Interface Speakable : oblige à implémenter la méthode speak()
```

```
interface Speakable {  
    public function speak();  
}
```

```
// Interface Flyable : oblige à implémenter la méthode fly()
```

```
interface Flyable {  
    public function fly();  
}
```

```
// Classe abstraite Animal : base pour les animaux avec une méthode abstraite move()
```

```
abstract class Animal {  
    // Méthode abstraite move()  
    abstract public function move();  
}
```

```
// Classe Dog héritant d'Animal et implémentant Speakable
```

```
class Dog extends Animal implements Speakable {  
    public function move() {  
        return "I run!";  
    }  
  
    public function speak() {  
        return "Woof!";  
    }  
}
```

```
// Classe Cat héritant d'Animal et implémentant Speakable
```

```
class Cat extends Animal implements Speakable {  
    public function move() {  
        return "I sneak!";  
    }  
  
    public function speak() {  
        return "Meow!";  
    }  
}
```

```
// Classe Bee implémentant Flyable (mais pas Speakable)
```

```
class Bee extends Animal implements Flyable {  
    public function move() {  
        return "I fly!";  
    }  
  
    public function fly() {  
        return "Buzzing through the air!";  
    }  
}
```

```

}

// Utilisation du polymorphisme avec l'interface Speakable
function makeAnimalSpeak(Speakable $animal) {
    echo get_class($animal) . " says: " . $animal->speak() . "\n";
}

// Utilisation du polymorphisme avec la méthode abstraite move() (pour n'importe
function moveAnimal(Animal $animal) {
    echo get_class($animal) . " moves like this: " . $animal->move() . "\n";
}

// Exemple d'utilisation
$dog = new Dog();
$cat = new Cat();
$bee = new Bee();

makeAnimalSpeak($dog); // Dog says: Woof!
makeAnimalSpeak($cat); // Cat says: Meow!
// makeAnimalSpeak($bee); // Provoquerait une erreur car Bee n'implémente pas Spe

moveAnimal($dog); // Dog moves like this: I run!
moveAnimal($cat); // Cat moves like this: I sneak!
moveAnimal($bee); // Bee moves like this: I fly!

// Pour les méthodes spécifiques aux interfaces Flyable
function makeAnimalFly(Flyable $animal) {
    echo get_class($animal) . " flies like this: " . $animal->fly() . "\n";
}

makeAnimalFly($bee); // Bee flies like this: Buzzing through the air!

?>

```

Avantages de combiner interface et abstraction

- **Flexibilité** : Les classes peuvent implémenter plusieurs interfaces et hériter de classes abstraites, offrant une grande flexibilité dans la conception des classes.
- **Clarté** : Les interfaces définissent un contrat clair pour les classes, tandis que les classes abstraites fournissent une implémentation de base pour les méthodes.
- **Modularité** : Les classes peuvent être conçues pour implémenter des interfaces spécifiques, ce qui permet de regrouper des fonctionnalités similaires et de les réutiliser dans d'autres classes.
- **Extensibilité** : Les classes peuvent facilement ajouter de nouvelles fonctionnalités en implémentant de nouvelles interfaces ou en héritant de nouvelles classes abstraites.

- **Séparation des préoccupations** : Les interfaces et les classes abstraites permettent de séparer les préoccupations et de maintenir un code propre et modulaire.
- **Facilité de test** : Les interfaces et les classes abstraites facilitent les tests unitaires en permettant de substituer des objets par des mocks ou des stubs.
- **Réutilisabilité** : Les interfaces et les classes abstraites favorisent la réutilisabilité du code en permettant de concevoir des classes génériques et extensibles.
- **Développement par contrat** : Les interfaces permettent de définir des contrats clairs entre les classes, ce qui facilite la collaboration entre les développeurs.
- **Dépendance minimale** : Les interfaces permettent de réduire les dépendances entre les classes en se concentrant sur les contrats plutôt que sur les implémentations.

Conclusion

Le polymorphisme permet de traiter des objets de différents types de manière uniforme, simplifiant le code et le rendant plus modulaire et extensible. En utilisant le polymorphisme, nous pouvons facilement ajouter de nouveaux types d'animaux sans avoir à modifier les fonctions existantes.

3. Les principes S.O.L.I.D.

Les principes SOLID sont un ensemble de cinq principes de conception orientée objet qui visent à rendre le code plus compréhensible, flexible et maintenable. Voici une explication détaillée de chaque principe :

Single Responsibility Principle (SRP)

- **Principe** : Une classe ne devrait avoir qu'une seule responsabilité, c'est-à-dire une seule raison de changer.
- **Explication** : Cela signifie que chaque classe doit se concentrer sur une seule tâche ou fonctionnalité. En suivant ce principe, le code devient plus facile à comprendre, à tester et à maintenir, car les changements dans une fonctionnalité n'affectent pas les autres.
- **Exemple** : Si une classe Report gère à la fois la génération de rapports et l'envoi de courriels, elle ne respecte pas le SRP. Il serait préférable de séparer ces responsabilités en deux classes distinctes, ReportGenerator et EmailSender.

Open/Closed Principle (OCP)

- **Principe** : Les entités du logiciel (classes, modules, fonctions, etc.) devraient être ouvertes à l'extension mais fermées à la modification.
- **Explication** : Cela signifie que le comportement d'une classe peut être étendu sans modifier son code source. Ce principe favorise l'utilisation de l'héritage ou de l'implémentation des interfaces pour ajouter de nouvelles fonctionnalités.
- **Exemple** : Si une classe Shape a des sous-classes Circle et Square, vous pouvez ajouter une nouvelle sous-classe Triangle sans modifier le code existant de Shape.

Liskov Substitution Principle (LSP)

- **Principe** : Les objets d'une classe dérivée doivent pouvoir être remplacés par des objets de la classe de base sans affecter la fonctionnalité du programme.
- **Explication** : Cela signifie que les sous-classes doivent être substituables à leurs classes de base sans que le code client ne doive connaître la différence. Pour respecter ce principe, les sous-classes ne doivent pas altérer le comportement attendu des méthodes de la classe de base.
- **Exemple** : Si une classe Bird a une méthode fly(), toutes les sous-classes comme Sparrow ou Eagle doivent pouvoir être utilisées sans problème à la place de Bird. Cependant, une sous-classe comme Penguin qui ne peut pas voler violerait ce principe.

Interface Segregation Principle (ISP)

- **Principe** : Les clients ne devraient pas être forcés à dépendre d'interfaces qu'ils n'utilisent pas.
- **Explication** : Cela signifie qu'il est préférable de créer plusieurs interfaces spécifiques à des clients plutôt qu'une seule interface générale. Ainsi, les classes n'implémentent que les méthodes dont elles ont besoin.
- **Exemple** : Si une interface Worker a des méthodes work() et eat(), une classe Robot qui implémente Worker ne devrait pas être obligée d'implémenter eat(). Il serait mieux de diviser l'interface en Workable avec la méthode work() et Eatable avec la méthode eat().

Dependency Inversion Principle (DIP)

- **Principe** : Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau. Les deux doivent dépendre des abstractions. Les abstractions ne doivent pas dépendre des détails. Les détails doivent dépendre des abstractions.
- **Explication** : Cela signifie que les classes de haut niveau (qui contiennent la logique d'application) ne doivent pas dépendre des classes de bas niveau (qui contiennent les détails de l'implémentation). Les deux devraient dépendre d'abstractions comme des

interfaces ou des classes abstraites. Ce principe favorise l'injection de dépendances pour inverser le contrôle de l'instanciation des objets.

- **Exemple :** Si une classe Service dépend d'une classe Repository, il est préférable de créer une interface IRepository que Service utilisera. Ainsi, Service dépend de l'abstraction IRepository, et vous pouvez facilement changer l'implémentation concrète de Repository sans modifier Service.

4. La Loi de demeter

La loi de Déméter, également appelée **principe du moindre couplage**, est une règle de conception en programmation orientée objet (POO) visant à réduire les dépendances entre les objets d'un système. Elle stipule qu'un objet ne doit interagir qu'avec les objets qui lui sont "proches" et éviter de "parcourir des chaînes" de dépendances.

Principes clés :

- Un objet doit uniquement appeler des méthodes de :
 - Lui-même
 - Ses attributs directs
 - Les objets créés localement (dans une méthode)
 - Les objets passés comme paramètres
 - Les objets retournés par ses propres méthodes
- Un objet ne doit pas :
 - Appeler des méthodes sur un objet retourné par une autre méthode (pas de "train de méthodes")
 - Dépendre de la structure interne d'autres objets

Exemple non conforme à la loi de Déméter :

```
$rue = $client->getProfil()->getAdresse()->getRue();
```

Dans cet exemple, l'objet `$client` accède à la méthode `getProfil()`, qui renvoie un objet. Ensuite, il appelle la méthode `getAdresse()` sur cet objet, qui renvoie un autre objet. Enfin, il appelle la méthode `getRue()` sur cet objet. Cela crée une chaîne de dépendances entre les objets, ce qui rend le code fragile et difficile à maintenir.

Exemple conforme à la loi de Déméter :

```
$rue = $client->getRue();
```

Dans cet exemple, l'objet `$client` a une méthode `getRue()` qui renvoie directement la rue de l'adresse du client. Cela respecte la loi de Déméter en évitant les chaînes de dépendances entre les objets.

4. Le principe d'Hollywood

Le principe d'Hollywood en programmation orientée objet (POO) se résume par la phrase : "Ne nous appelez pas, nous vous appellerons."

Ce principe vise à inverser le contrôle du flux d'exécution. Plutôt que de laisser les classes "basses" ou spécifiques diriger les interactions, ce sont les classes "hautes" ou générales (souvent des frameworks ou des gestionnaires) qui contrôlent le flux en appelant les classes spécifiques.

Principes clés :

- Inversion du contrôle :
 - Les objets ne décident pas eux-mêmes quand et comment leur logique sera utilisée. Ils s'enregistrent auprès d'une entité supérieure qui les appelle au moment opportun.
- Encapsulation des responsabilités :
 - Le principe favorise une architecture où les modules de haut niveau dirigent les interactions avec les modules de bas niveau.

Exemple sans le principe d'Hollywood :

```
class RegistrationService
{
    public function register(User $user): void
    {
        // Logique d'inscription (sauvegarde en base de données)
        echo "Utilisateur enregistré : " . $user->getEmail() . "\n";

        // Envoi d'un email de bienvenue
        $this->sendWelcomeEmail($user);

        // Ajout à la liste marketing
        $this->addToMarketingList($user);
    }
}
```

```

private function sendWelcomeEmail(User $user): void
{
    echo "Email de bienvenue envoyé à : " . $user->getEmail() . "\n";
}

private function addToMarketingList(User $user): void
{
    echo "Utilisateur ajouté à la liste marketing : " . $user->getEmail() . "
}
}

```

Dans cet exemple, la classe `RegistrationService` est responsable de l'ensemble du processus d'inscription, y compris l'envoi d'un email de bienvenue et l'ajout à la liste marketing. Cela crée une forte dépendance entre `RegistrationService` et les classes `EmailService` et `MarketingService`. Si l'une de ces classes change, `RegistrationService` devra également être modifié.

Exemple avec le principe d'Hollywood :

```

// Gestion de l'inscription
class RegistrationService
{
    public function __construct(private EventDispatcher $dispatcher) {}

    public function register(User $user): void
    {
        // Logique d'inscription (sauvegarde en base de données)
        echo "Utilisateur enregistré : " . $user->getEmail() . "\n";

        // Déclenchement de l'événement
        $this->dispatcher->dispatch(new UserRegisteredEvent($user));
    }
}

// Listener pour l'email
class WelcomeEmailListener
{
    public function onUserRegistered(UserRegisteredEvent $event): void
    {
        echo "Email de bienvenue envoyé à : " . $event->getUser()->getEmail() . "
    }
}

// Listener pour le marketing

```

```
class MarketingSubscriber
{
    public function onUserRegistered(UserRegisteredEvent $event): void
    {
        echo "Utilisateur ajouté à la liste marketing : " . $event->getUser()->ge
    }
}
```

Dans cet exemple, la classe `RegistrationService` est responsable de l'inscription des utilisateurs et déclenche un événement `UserRegisteredEvent`. Les classes `WelcomeEmailListener` et `MarketingSubscriber` écoutent cet événement et réalisent les actions appropriées. Cela permet de découpler les différentes étapes du processus d'inscription et de rendre le code plus modulaire et flexible. Si une nouvelle action doit être ajoutée, il suffit de créer un nouveau listener sans modifier `RegistrationService`.

